NXTLVLSOCIAL.AI

FROM THE 03:07 AM REEL

The setup that *saved him.*

Prod Guard for Claude Code. Claude snapshots your database before every destructive command, and refuses to touch production at all.

5-minute install | Tested on PostgreSQL 16 | Free to use



A DROP DATABASE at 3 AM should cost you a minute, not your company.

WHAT HAPPENED AT 03:07

He thought he was wiping the test database. He was connected to production. 1.2 million rows, gone, and the nightly backup had quietly died in March. The only copy was a snapshot his AI agent took first.

This kit makes that snapshot automatic. Five locks, all plain files in your repo:

1. **Snapshot hook**
   Destructive SQL anywhere? Claude takes a snapshot first, then asks you to confirm.
2. **Production refusal**
   Destructive SQL aimed at production is refused outright. A human runs it, or nobody does.
3. **House rules**
   CLAUDE.md tells Claude to name the database it is on and never touch production.
4. **Backup alarm**
   Every session starts by checking your last backup is fresh. No more "stopped in March".
5. **Production lock**
   A read-only role for everyday access, and a red PRODUCTION prompt in psql.

YOU NEED

- Claude Code
- bash and **jq**
- PostgreSQL client tools (**pg_dump**, **psql**)

install

```
# macOS
brew install jq libpq && brew link --force libpq
# Ubuntu / Debian
sudo apt install jq postgresql-client
```

WHERE EVERYTHING GOES

project layout

```
your-project/
├── CLAUDE.md          rules (3)
├── .gitignore         (2)
└── .claude/
    ├── settings.json  hooks (2)
    ├── prod-guard.env URLs (2)
    └── hooks/
        ├── prod-guard.sh    (1)
        └── check-backups.sh (4)
```

01 The snapshot *hook*

A PreToolUse hook: Claude Code runs it before every Bash command Claude wants to run. Normal commands pass straight through. Destructive SQL on production is denied. Anywhere else, the database is dumped to `.snapshots/` first, and you're asked to confirm. If the snapshot fails, the command doesn't run.

.claude/hooks/prod-guard.sh

```
#!/usr/bin/env bash
# Prod Guard: a Claude Code PreToolUse hook for the Bash tool.
#   destructive SQL on production → refused
#   destructive SQL anywhere else → snapshot first, then you confirm
set -uo pipefail

cmd=$(jq -r '.tool_input.command // ""')
dir="${CLAUDE_PROJECT_DIR:-.}"
[ -f "$dir/.claude/prod-guard.env" ] && . "$dir/.claude/prod-guard.env"

decide() {  # decide allow|ask|deny "reason"
  jq -n --arg d "$1" --arg r "$2" '{hookSpecificOutput: {hookEventName: "PreToolUse",
    permissionDecision: $d, permissionDecisionReason: $r}}'
  exit 0
}

# 1. Is it destructive? If not, the normal permission flow applies.
shopt -s nocasematch
drop='drop[[:space:]]+(database|schema|table|owned)|truncate[[:space:]]|dropdb[[:space:]]'
reset='migrate[[:space:]]+reset|db:(drop|reset)'
wipe=no
[[ "$cmd" =~ $drop|$reset ]] && wipe=yes
[[ "$cmd" =~ delete[[:space:]]+from && ! "$cmd" =~ where ]] && wipe=yes
[ "$wipe" = no ] && exit 0

# 2. Production? Claude never runs it.
prod="${PROD_PATTERN:-(^|[^a-z])(prod|production)([^a-z]|$)}"
if [[ "$cmd ${SNAPSHOT_DB_URL:-}" =~ $prod ]]; then
  decide deny "Prod Guard: destructive command on production. Hand it to a human."
fi

# 3. Anywhere else: snapshot first, or it doesn't run.
if [ -z "${SNAPSHOT_DB_URL:-}" ]; then
  echo "Prod Guard: no SNAPSHOT_DB_URL in .claude/prod-guard.env. Blocked." >&2
  exit 2
fi
mkdir -p "$dir/.snapshots"
file="$dir/.snapshots/snapshot_$(date +%Y%m%d_%H%M%S)_$$.dump"
if ! pg_dump -Fc -f "$file" "$SNAPSHOT_DB_URL" 2>"$dir/.snapshots/error.log"; then
  echo "Prod Guard: snapshot failed (.snapshots/error.log). Blocked." >&2
  exit 2
fi
decide ask "Prod Guard: snapshot saved to .snapshots/$(basename "$file"). Run it?"
```

terminal

```
mkdir -p .claude/hooks
# save the script above as .claude/hooks/prod-guard.sh, then:
chmod +x .claude/hooks/prod-guard.sh
```

02 Wire it *in*

Register the hooks and add two permission guard rails. Deny always wins in Claude Code: `dropdb` is blocked for good, and every `psql` call asks you first.

.claude/settings.json

```
{
  "permissions": {
    "deny": ["Bash(dropdb *)", "Read(./.env.production)"],
    "ask": ["Bash(psql *)", "Bash(pg_restore *)"]
  },
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "\"${CLAUDE_PROJECT_DIR}\"/.claude/hooks/prod-guard.sh"
      }]
    }],
    "SessionStart": [{
      "hooks": [{
        "type": "command",
        "command": "\"${CLAUDE_PROJECT_DIR}\"/.claude/hooks/check-backups.sh"
      }]
    }]
  }
}
```

.claude/prod-guard.env

```
# .claude/prod-guard.env (keep it out of git)
# The database Claude works on. Never production.
SNAPSHOT_DB_URL="postgres://localhost:5432/app_dev"
# Where nightly backups land; how old is too old.
BACKUP_DIR="/var/backups/postgres"
MAX_AGE_HOURS=26
```

terminal

```
echo ".claude/prod-guard.env" >> .gitignore
echo ".snapshots/" >> .gitignore
```

Point `SNAPSHOT_DB_URL` at the database Claude works on. If it ever contains "prod", the hook refuses every destructive command, which is what you want.

03 The house *rules*

Hooks are the hard stop. These lines in CLAUDE.md make Claude careful before it ever gets there.

CLAUDE.md (add to yours)

```
## Database safety (Prod Guard)

- Before any destructive database change (DROP, TRUNCATE, DELETE without
  WHERE, migrate reset) a snapshot is taken first. The prod-guard hook does
  this. Never try to get around it.
- Say which database you are connected to before you run any SQL.
- Never run commands against production. If a task needs production, stop
  and give the exact command to the human.
- If the session-start message says backups are stale, say so first.
```

04 The backup *alarm*

His backup job died in March and nobody looked. This runs at the start of every Claude Code session (the SessionStart hook from step 2), and what it prints goes straight into Claude's context: a stale backup is the first thing you hear about.

.claude/hooks/check-backups.sh

```
#!/usr/bin/env bash
# Prod Guard: runs when a Claude Code session starts.
# What it prints goes into Claude's context.
dir="${CLAUDE_PROJECT_DIR:-.}"
[ -f "$dir/.claude/prod-guard.env" ] && . "$dir/.claude/prod-guard.env"
backups="${BACKUP_DIR:-/var/backups/postgres}"
max="${MAX_AGE_HOURS:-26}"

if [ -n "$(find "$backups" -type f -mmin "-$((max * 60))" 2>/dev/null | head -n 1)" ]; then
  echo "Prod Guard: backups OK (one newer than ${max}h in $backups)."
else
  echo "Prod Guard WARNING: no backup newer than ${max}h in $backups."
  echo "Treat production as having NO backup. Tell the user before anything else."
fi
exit 0
```

terminal

```
chmod +x .claude/hooks/check-backups.sh
```

05 Lock *production*

The last line of defence is the database itself. Everyday production access goes through a read-only role: it can read everything and delete nothing. We tested it: DELETE, TRUNCATE, DROP TABLE and DROP DATABASE are all refused.

readonly-role.sql (run once, as owner)

```
-- Run once on production, as the owner. Day-to-day, connect as dev_readonly.
CREATE ROLE dev_readonly LOGIN PASSWORD 'change-me';
GRANT CONNECT ON DATABASE app TO dev_readonly;
GRANT USAGE ON SCHEMA public TO dev_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO dev_readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO dev_readonly;
```

~/.psqlrc-prod

```
-- ~/.psqlrc-prod: a red PRODUCTION prompt, and nothing sticks until you type COMMIT
-- (it can't stop DROP DATABASE: use the read-only role for that)
\set PROMPT1 '%[%033[1;97;41m%] PRODUCTION %[%033[0m%] %/%R%# '
\set AUTOCOMMIT off
\set ON_ERROR_ROLLBACK interactive
```

~/.zshrc or ~/.bashrc

```
alias psql-prod='PSQLRC=~/.psqlrc-prod psql "$PROD_DATABASE_URL"'
```

Use the dev_readonly login in `PROD_DATABASE_URL`. The red prompt plus `AUTOCOMMIT off` means a stray DELETE or DROP TABLE is rolled back unless you type COMMIT. It can't stop DROP DATABASE (psql runs that outside a transaction), which is why the read-only role comes first.

06 Test it in *60 seconds*

Feed the hook exactly what Claude Code sends it. You should get **ask** plus a new file in `.snapshots/` for the first, and **deny** for the second.

terminal

```
echo '{"tool_input":{"command":"psql -c \"DROP TABLE users\""}}' \
  | CLAUDE_PROJECT_DIR=$PWD .claude/hooks/prod-guard.sh

echo '{"tool_input":{"command":"psql -h prod-db -c \"DROP TABLE users\""}}' \
  | CLAUDE_PROJECT_DIR=$PWD .claude/hooks/prod-guard.sh
```

IF IT HAPPENS ANYWAY

restore

```
# rows or a table gone
pg_restore --clean --if-exists \
  -d "$SNAPSHOT_DB_URL" .snapshots/<file>.dump

# the whole database gone
createdb app_dev
pg_restore \
  -d "$SNAPSHOT_DB_URL" .snapshots/<file>.dump
```

Both tested: 1,000 rows deleted, then a dropped database, restored to 1,000 rows each time.

CHECKLIST

- [ ] Hook scripts saved in .claude/hooks/ and executable
- [ ] settings.json in place
- [ ] prod-guard.env filled in, and in .gitignore with .snapshots/
- [ ] Rules added to CLAUDE.md
- [ ] The test above says "ask" (with a snapshot) and "deny" on prod
- [ ] Nightly backups land in BACKUP_DIR
- [ ] Everyday production access uses dev_readonly



A seatbelt, not a backup plan.

Snapshots protect the database Claude works on. Production still needs real, monitored backups, and now you'll know the day they stop. Follow **NXTLVLSOCIAL.AI** for more Claude setups like this one.